

Core console APIs

When to use this

Start here whenever you need structured console output or simple input helpers without adopting the view or routine systems. These APIs sit closest to `System.Console` and can be introduced incrementally into existing apps.

Key types

- `Consoul.Write` – Primary output helper that respects `RenderOptions.WriteMode` and color schemes.
- `RenderOptions` – Global configuration for default colors, write modes, prompt text, and invalid message styles.
- `ColorScheme` – Bundles foreground/background combinations for repeated use.
- `Consoul.Wait` – Displays a “Press enter to continue” style message and pauses execution.
- `Consoul.Input` / `Consoul.Ask` / `Consoul.Read` – Input helpers for free-form text, boolean confirmations, and asynchronous reading.

Minimal example

```
using ConsoulLibrary;

RenderOptions.DefaultScheme = new ColorScheme(ConsoleColor.White, ConsoleColor.Black);
RenderOptions.InvalidScheme = new ColorScheme(ConsoleColor.White, ConsoleColor.DarkRed);

Consoul.Write("System initialised", RenderOptions.DefaultScheme);
if (!Consoul.Ask("Proceed with deployment?"))
{
    Consoul.Write("Operation canceled", ConsoleColor.Yellow);
    return;
}

string environment = Consoul.Input("Target environment:");
Consoul.Write($"Deploying to {environment}", ConsoleColor.Green);
```

Advanced tips

- **Write modes & fallbacks** – `RenderOptions.WriteMode` lets you suppress all output or only colourised output. This is useful when running tests or logging to CI pipelines. When authoring custom writers, call `RenderOptions.GetColorOrDefault` to respect the active mode automatically.

- **Scoped overrides** – Clone `RenderOptions` into a local variable, tweak colours or padding, and restore it after calling `Consoul.Write` to emulate Spectre-style themes for specific sections.
- **Custom continue prompts** – Override `RenderOptions.ContinueMessage` to localise or rephrase the pause text used by `Consoul.Wait`. Adjust `RenderOptions.SubnoteScheme` if you want the pause prompt to use a unique colour without altering other helpers.
- **Integration with logging** – Register `ConsoulLoggerProvider` in a `HostApplicationBuilder` to emit structured log messages through Consoul's formatting. Update `ConsoulLogger.LogLevelToColorMap` if you prefer different colours per severity.
- **Testing** – Pass alternative `TextWriter/TextReader` implementations into `Console.SetOut / Console.SetIn` when unit testing methods that rely on Consoul helpers. Combine this with dependency injection to provide fake timers or cancellation tokens for `Consoul.Read` scenarios.